An Improved Algorithm for String Matching using Index Based Shifting Approach

Tania Islam
Computer Science and Engineering Discipline
Khulna University
Khulna-9208, Bangladesh.
Email: tania.bd.09@gmail.com

Kamrul Hasan Talukder
Computer Science and Engineering Discipline
Khulna University
Khulna-9208, Bangladesh.
Email: k.h.t@alumni.nus.edu.sg

***Abstract*—Bioinformatics is now considered as one of the most studied branches in biological science which deals with the exploring of different kinds of methods for analyzing, storing and retrieving biological data, such as protein sequences and nucleic acid (DNA/RNA), configurations, functions, pathways, and genetic relations. In bioinformatics, genomic sequence analysis has led to the evolution of well-organized pattern discovery algorithms to produce search over large DNA sequences. The most important factor for efficiency of a matching algorithm is depending on the total number of character comparisons. Index Based Shift (IBS) algorithm which is the name chosen for our proposed algorithm was tested on different length of DNA sequences data set. The IBS algorithm shows great efficiency in terms of total number of character comparisons it requires than that of ABSBMH algorithm. The experimental result shows that our proposed IBS algorithm works better when length of pattern is increased.**

***Index Terms*—string matching, pattern matching, exact string matching, biological sequence.**

I. Introduction

String matching which is also define as string searching is found as the most important and traditional problem in the field of computer science. Single or multiple patterns are to be searched within any given string. It plays a key part in various real world problems and applications.

A few of its lucrative applications are bioinformatics, spam filtering, encroachment identification system, digital scientific test, plagiarism identification, spell checker and correction, search engines, and information recovery systems etc [1]. With the beginning of next generation sequence technologies, there are lots of genomic sequences of entities of the same species presented. A very small amount of differences are found to each other. So, a highly efficient pattern matching algorithm is very much required in such specific DNA sequence. String matching aimed to find a piece of text called pattern P= $p_0.p_1.p_2..p_{m-1}$, in a given text T= $t_0.t_1.t_2..t_{n-2}$ t_{n-1}, where n is always larger than m and p_i, t_i are symbols of the alphabet.

String matching algorithms can be divided into two types, single pattern matching and multiple pattern matching. In single pattern matching only one pattern is to be searched into the given text whereas in the multiple patterns matching multiple patterns are given to be searched. Again many existing pattern matching algorithms are classified into two categories: exact string matching algorithm and approximate string matching algorithm [2]. In exact pattern matching, the pattern P will match 100% with the given text T, whereas in the approximate pattern matching a few part of pattern P will match with the given text T. Approximate string matching is important for identifying sequencing errors or mutations among individuals or polymorphisms. Exact single pattern matching algorithm can be further classified into two categories: on-line and off-line string matching algorithm [3]. In the online string matching algorithm only the given pattern will be pre-processed and the given text will not be pre-processed. Again in off-line string matching algorithm only given text will be pre-processed; whereas, the given pattern will not be processed that means it leaves intact. In this paper, we focused on the off-line string matching algorithm where we will try to find a solution of overall existing algorithm by minimizing the number of attempts it requires to match the pattern with given string.

The organization of the paper is as follows. In section II, we have described some related exact string matching algorithms. Section III explains the proposed methodology. Section IV defines the experimental result analysis including comparisons with other algorithms. Finally, section V concludes the paper with some future direction.

II. Related Work

DNA or deoxyribonucleic acid is the genetic component in humans species and almost all other creature. It is the combination of four characters, A (Adenine), G (Guanine), C (Cytosine) and T (Thiamine). A character set $\sum$ = $\{A,C,G,T\}$ is the set of alphabet for DNA sequence which are used in this algorithm. Both given string and pattern are combination of those four characters [8][10].

The following symbols are used in our proposed algorithm:

DNA sequence which contains characters is $\sum$ = $\{A,C,G,T\}$.

$S[n]$ defines text length of n.

ϕ declare the null string.

$P[m]$defines a pattern which is length of m [8].

The most oldest and initial algorithm for string searching is the Brute-force algorithm. But it is the least efficient to check a specific pattern in a given string. This algorithm works

978-1-5386-1150-0/17/$31.00 2017 IEEE

by matching with given text that is T= $t_0.t_1.t_2. ..t_{n-2}.t_{n-1}$ with given pattern that is P = $p_0.p_1.p_2...p_{m-1}$ in character by character. That means when $t[0]$ is not equal to $p[0]$ then it starts it matching with $t[1]$ and $p[0]$. And this process is continued until any mismatch is found. After finding the complete match of the pattern it gives back the position of the pattern and continues its process of next character. The average case and worst case run time complexity of the algorithm are O(m+n) and O(n/m).

Various algorithms are developed to improve the Brute-force algorithm; Boyer-Moore (BM) [4] is one of them. BM algorithm is considered as one of the best algorithms for a long period of time. It contains two phases that is preprocessing phase and searching phase. In preprocessing phase the pattern characters are stored in a bad character table by using (1),

$$value = \text{Length(of the pattern)} - \text{Index(of character)} \quad (1)$$

It also uses two recomputed functions to shift the window that is good suffix shift and bad character shift. The runtime complexity for best case is (n/m).

The simplification of BM algorithm is Horspool Algorithm [5]. The main difference between BM algorithm and this algorithm is missing of good suffix shift table. It only uses bad character table. When any match or mismatch occurs the algorithms shift the text window to the right side according to the value of bad character table. The best case time complexity for this algorithm is $O(m/n)$.

Maximum-Shift algorithm [6] is proposed by Kadhim et al. This algorithm is a hybrid algorithm because, it is the combination of three algorithms: Horspool algorithm, Zuh-Takaoka algorithm and Quick-Search algorithm. The preprocessing phase of this algorithm is the combination of QS and ZT algorithm whereas its searching phase is equal to the modified Horspool algorithm proposed by Liu [7] because, this algorithm checks last two characters of the text window instead of one character.

ABSBMH Algorithm [9] is the combination of SSABS algorithm and Quick search algorithm. In this algorithm the author uses quick search bad character function in pre-processing stage and starts searching using modified Horspool algorithm. They compare last two character of text instead of one character. This algorithm gives worst result when using DNA sequence data set.

III. Proposed Algorithm

This section concentrates on solving exact string matching algorithm. Our proposed algorithm consists of two phases, one is pre-processing phase and other is searching phase.

A. Pre-processing Phase

Preprocessing phase is used to preprocess the pattern or given string which can minimize the searching time. Here we preprocess the given string in order to minimize the number of attempts and number of character comparisons.

The proposed algorithm uses the indices value of given string/DNA sequence for pre-processing.

Pre-processing phase is completed in two stages.

1) Pre-process the given input string: At first it tries to know the all indices of given string. We work with DNA sequence and it only contains four characters $\{A, G, C, T\}$ [8][10]. For this purpose we have to maintain four array indices table.

In our proposed IBS algorithm, we used ASCII indexing technique from [8] to define the subscript value of each character. The ASCII indexing technique is considered this formula, [((S[i])-64)%5]

TABLE I
Subscript Values for DNA Characters

SL No.	DNA	ASCII Value	ASCII Value-64	(ASCII Value-64)%5	Array Subscript
1	A	65	1	1	1
2	C	67	3	3	3
3	G	71	7	2	2
4	T	84	20	0	0

By using this formula we always get value 0,1,2,3 and 4 of the characters. This subscript value are using in the two-dimensional array.

Let the string S be with length of n and Pattern be P with length of m .Suppose we have a string like AGCCTTTC-GACC, which is considered input string. Then we have to construct the following index table II.

TABLE II
Selected Indices Value for DNA Sequence

DNA	Sequence Indices				
T[0]	4	5	6		
A[1]	0	9			
G[2]	1	8			
C[3]	2	3	7	10	11

2) Computing substring: In second stage of pre-processing phase we have to divide the given input string into different number of substrings. The length of substring is equal to the length of pattern and also the first character of substring is equal to the first character of pattern. It should follow the following criteria:

- The substring length is exactly equal to pattern length.
- The process starts to compute substring from the text that matches with the patterns first character as we already know the index value of text.
- Last substring indices will generate according to the following (2).

$$\text{Substring length} = S[n-1] - (P[m]-1) \quad (2)$$

For the above-given string when the pattern, P= TTT.
The substrings for the given string are given in table III.

B. Searching Phase

In our proposed method, our searching phase will follow the following step:

- In first step, matching started in second character of pattern with the second character of first substring from

TABLE III
INDICES OF COMPUTING SUBSTING

DNA substring	Indices of substring
TTT	4
TTC	5
TCG	6

left to right because of substring first character and pattern first character always same.

- If they match, go to the third character of both pattern and substring.
- If there is any mismatch, it skips matching that substring and pattern window goes to the next substring and continue the process.
- If any match found, it returns the location of that pattern in the given string.
- There are repetitions of pattern in the given string. For that when one match is found it will continue its process until all substrings are matched.
- This process is stopped when all substrings are matched.

By doing this process, our proposed IBS algorithm is capable of reducing the number of character comparisons to find a given pattern in a given string.

C. Working Example

In this section, we give a working example to explain our proposed IBS algorithm.

Given string,

S= ATCTAACATCATAACCCTGCAGAGAGGAT

Given Pattern,

P=GCAGAGAG

At first stage, it pre-processes the given string by storing it indices into a two-dimensional array. After storing the indices of all character of given text, the value of indices is given in table IV.

TABLE IV
INDICES OF GIVEN STRING

DNA	Indices										
T	1	3	8	11	17	28					
A	0	4	5	7	10	12	13	20	22	24	27
C	2	6	9	14	15	16	19				
G	18	21	23	25	26						

Now it divides the string into different substrings. Before dividing the text into substrings, it checks the 1^{st} character of pattern. In this given pattern, it starts with character G. So all the substring should start with G and also the length of substring is equal to the length of pattern. By following this procedure, our computed substring is given in table V.

TABLE V
INDICES OF COMPUTING SUBSTING

Substring	Indexes
GCAGAGAG	18
GAGAGGAT	21

In this process it is not considered 23, 25, and 26 number of position of character G, because, if we consider them the length of substring will exceed the length of pattern.

Now our pre-processing stage is completed. So we start our matching stage.

1^{st} **attempt**

At first, we compare from left to right of the pattern and 1^{st} substring from the second character of the pattern and second character of substring. Its a match.

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

So continue our matching from left to right direction at the end of the pattern and substring.

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

G	C	A	G	A	G	A	G
G	C	A	G	A	G	A	G

Now its a complete match, so the number of matching indices is 18.

2^{nd} **attempt**

We compare from left to right of the pattern and substring. We did not compare the first character because we already know that the both characters are equal.

G	A	G	A	G	G	A	T
G	C	A	G	A	G	A	G

Now, we compare with the second character of substring with the second character of pattern that is $A \neq C$ and skips this substring. So our pattern window shifts to the next the substring.

Finally, there is no more substring left. If there is more substring left, we continue our process to find the repetitions of the pattern occur in given string.

So, the number of character comparison is 8 and number of attempt is 2.

IV. Experimental Result

We use only DNA sequences dataset [9] as our experimental data. This type involves nucleotides sequences with Adenine, Guanine, Cytosine and Thiamin [10]. We use C# programming language to implement our proposed algorithm.

There are two common features that are used to evaluate the performance of string matching algorithms with different types of applications and these measures are listed below [6]:

A. Number of character comparisons

This factor mainly refers to the actual comparisons that are occurring between given the text and a pattern. The algorithms with fewer character comparisons are considered as better.

B. Number of attempt

This factor mainly considers as the distance that a pattern moves along with given string or text. Obviously, the performance of algorithm is better when the number of attempts is less.

We compare our proposed Index Based Shift algorithm with the ABSBMH algorithm [9] in terms a number of character comparison.

For experimental analysis, we use pattern size of 12, 24, 36, 48, 60, 72, 84 and 96, with the 50MB size of given input [9]. The experimental result is obtained by using this data set shown in Table VI and in Figure 1. From table VI and fig 1, it can clearly say that our proposed algorithm works better than ABSBMH [9] algorithm.

TABLE VI
Number of Character Comparison Using DNA Sequence

Pattern Length	ABSBMH	IBS
12	24250524	11008707
24	33727763	15190090
36	18408266	11025663
48	17080181	11008695
60	30081174	11008690
72	25769929	11008688
96	18154013	11008788

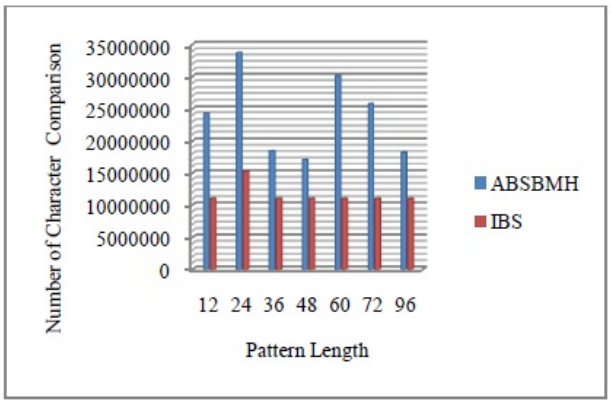

Fig. 1. Number of Character Comparison Using DNASequence

V. Conclusion

String matching is a very well-known topic in computer science since it occupies a wide space in computer application. We have presented a new efficient algorithm which is known as IBS algorithm. This algorithm works faster and gives a better result when the number of character repetitions is limited and length of the pattern is long. With the increasing of repetitions of character it also increases the number of substring. This algorithm gives worst result when using English text and Protein sequence but gives better output using DNA sequences.

Acknowledgment

We would like to acknowledge the ICT Division of Ministry of Posts, Telecommunications and Information Technology, Government of Peoples Republic of Bangladesh and also Computer Science and Engineering Discipline, Khulna University for supporting this work.

References

[1] V. SaiKrishna, A. Rasool and N. Khare,"String Matching and its Applications in Diversified Fields, IJCSI International Journal of Computer Science Issues, Vol. 9, Issue 1, No 1, January 2012.
[2] N. Gonzalo, "A guided tour to approximate string matching, ACM Computing Surveys (CSUR).33(1):31-88,2001.
[3] P.D.Michailidis, K.G.Margaritis, " On-line string matching algorithms: Survey and experimental results, International Journal of Computer Mathematics, pp. 740-741, March 2007.
[4] R.S. Boyer, and J.S. Moore, "A fast string searching algorithm,"Communications of the ACM ,pp. 762-772,october 1977.
[5] A. Jens," Exact pattern matching: Adapting the boyer-moore algorithm for DNA searches , Peer J Pre Prints, February 2016.
[6] H. A. Kadhil, N. A. A. Rashid, "Maximum-shift string matching algorithms, Computer and Information Sciences, International Conference on. IEEE, 2014.
[7] Y. Liu, "Comparison studies of exact pattern matching algorithms, Technical Report, Computational Biology Algorithm, Department of Computer Science, University of Victoria, December 2008.
[8] R. Bhukya and D. Somayajulu , "An Index Based Sequential Multiple Pattern Matching Algorithm Using Least Count, International Conference on Life Science and Technology, IACSIT Press, Singapore, vol.3, 2011.
[9] S. S. Mahmood, A. Dabbagh1, and N. H. Barnouti, "A New Efficient Hybrid String Matching Algorithm to Solve the Exact String Matching Problem, British Journal of Mathematics and Computer Science, pp. 1-14, 2017.
[10] G. Cai, X. Nie and Y. Huang, "A Fast Hybrid Pattern Matching Algorithm for Biological Sequences, 2nd International Conference on Biomedical Engineering and Informatics, October 2009.